Mini Project Report

Project Title: Linux System Resource Monitoring Dashboard

Team Members: SIMI MALLICK- 27631725022

KUNAL DAS- 27631725018

CHANDAN PAL- 27631725023

GOURAB PAN- 27631725020

RAUNAK KUMAR SINGH- 27631725019

Faculty: DR. MUNSHI YUSUF ALAM

Semester:- FIFTH



Department of Computer Science & Engineering
(CYBER SECURITY)

Budge Budge Institute of Technology (BBIT)



Certificate

This is to certify that the mini-project entitled "Linux System Resource Monitoring Dashboard" has been successfully completed by the following students:

SIMI MALLICK – 27631725022
KUNAL DAS – 27631725018
CHANDAN PAL – 27631725023
GOURAB PAN – 27631725020
RAUNAK KUMAR SINGH – 27631725019

Under the guidance of **DR. MUNSHI YUSUF ALAM**

at Department of Computer Science & Engineering (Cyber Security), Budge Budge Institute of Technology, Kolkata

Guide/Faculty Name & Signature: _____

Head of Department: _____

Date: _____ **Seal:**

_____

Acknowledgement

We would like to express our sincere gratitude to our project guide, **Dr. Munshi Yusuf Alam**, and the Department of Computer Science & Engineering (Cyber Security), Budge Budge Institute of Technology, for their valuable guidance, encouragement, and continuous support throughout the completion of this mini project.

We are thankful to all faculty members, friends, and team members who directly or indirectly helped us in completing this project successfully. Their suggestions and motivation greatly improved our understanding of Linux system monitoring and operating system concepts such as process management, CPU management, and memory management.

Finally, we would like to thank our institution for providing us with the necessary facilities and resources to complete this project successfully.

Signature of Team Members:

Simi Mallick (27631725022) _____

Kunal Das (27631725018) _____

Chandan Pal (27631725023) _____

Gourab Pan (27631725020) _____

Raunak Kumar Singh (27631725019) _____

1. Abstract

The Linux System Resource Monitoring Dashboard is a Python-based application that monitors and displays the current resource utilization of a Linux computer. It collects information about CPU usage, memory consumption, disk usage, disk I/O, and running processes.

The dashboard identifies processes that consume high amounts of CPU or memory. It helps users understand system performance, detect resource-intensive applications, and analyze resource utilization. The project demonstrates important Operating System concepts, including process management, memory management, file systems, process monitoring, and system resource allocation.

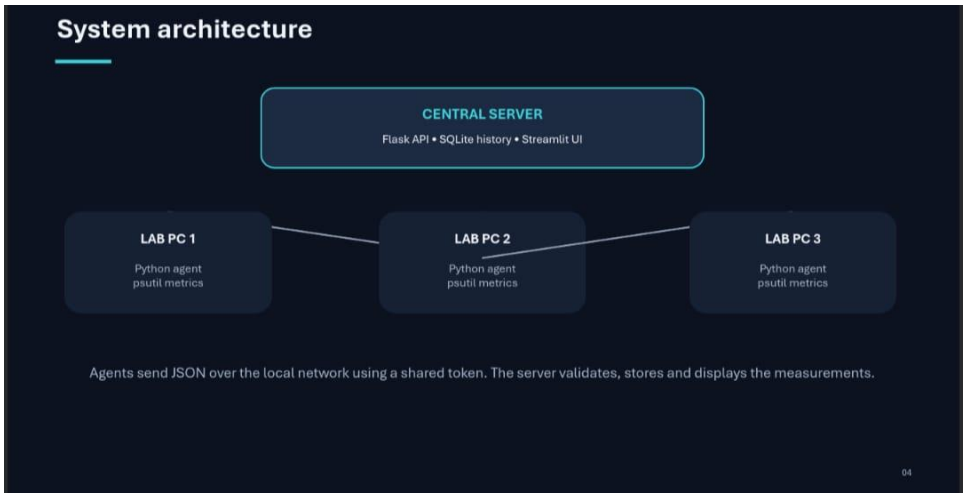
2.Objectives

1. Monitor real-time CPU utilization.
2. Display total, used, and available RAM.
3. Monitor disk capacity and storage utilization.
4. Display disk read/write activity.
5. List running processes and their resource consumption.
6. Identify high-CPU and high-memory processes.
7. Refresh monitoring information periodically.
8. Present system statistics in a simple dashboard.

3.OS concepts used

OS concept	Application in this project
Processes	Lists running processes
CPU management	Measures CPU utilization
Memory management	Monitors RAM usage and availability
File systems	Checks disk capacity and free space
Disk I/O	Measures disk read/write activity
System monitoring statistics	Collects and displays resource

4. System Architecture



5. Algorithm

1. Start the program.
2. Import the required Python libraries.
3. Collect CPU utilization information.
4. Collect total, used, and available memory.
5. Collect disk utilization and disk I/O statistics.
6. Retrieve information about running processes.
7. Sort processes by CPU and memory usage.
8. Display system statistics and resource-intensive processes.
9. Wait for a short interval and refresh the dashboard.
10. Stop when the user presses Ctrl + C.

6.Python Source Code

```python
import os
import time
import platform

def show_dashboard():
    os.system("cls" if os.name == "nt" else "clear")

    print("=" * 50)
    print(" LINUX SYSTEM RESOURCE MONITORING DASHBOARD")
    print("=" * 50)

    print("Operating System:", platform.system())
    print("Computer Name:", platform.node())

    # CPU Information
    try:
        with open("/proc/stat", "r") as file:
            cpu = file.readline().split()
            values = list(map(int, cpu[1:]))
            idle = values[3]
            total = sum(values)

        time.sleep(1)

        with open("/proc/stat", "r") as file:
            cpu = file.readline().split()
            values2 = list(map(int, cpu[1:]))
            idle2 = values2[3]
            total2 = sum(values2)

        usage = 100 * (1 - (idle2 - idle) / (total2 - total))
        print(f"\nCPU Usage: {usage:.2f}%")
    except (OSError, ValueError, ZeroDivisionError):
        print("\nCPU Usage: Not available")
```



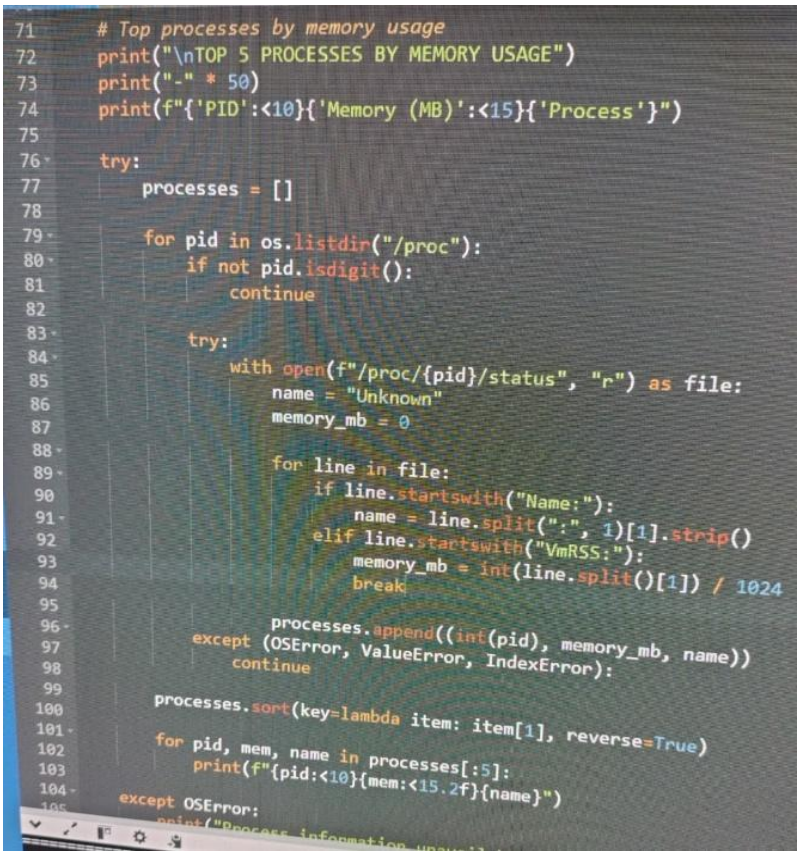
```python
    # Memory Information
    try:
        memory = {}
        with open("/proc/meminfo", "r") as file:
            for line in file:
                parts = line.split()
                memory[parts[0].rstrip(":")] = int(parts[1])

        total_ram = memory["MemTotal"]
        available_ram = memory["MemAvailable"]
        used_ram = total_ram - available_ram
        percentage = used_ram / total_ram * 100

        print(f"Total RAM: {total_ram / 1024:.2f} MB")
        print(f"Used RAM: {used_ram / 1024:.2f} MB")
        print(f"Available RAM: {available_ram / 1024:.2f} MB")
        print(f"Memory Usage: {percentage:.2f}%")
    except (OSError, KeyError, ValueError, ZeroDivisionError):
        print("Memory information unavailable")

    # Disk Information
    try:
        stats = os.statvfs("/")
        total_disk = stats.f_blocks * stats.f_frsize
        free_disk = stats.f_bavail * stats.f_frsize
        used_disk = total_disk - free_disk
        disk_usage = used_disk / total_disk * 100

        print(f"\nTotal Disk: {total_disk / (1024**3):.2f} GB")
        print(f"Used Disk: {used_disk / (1024**3):.2f} GB")
        print(f"Free Disk: {free_disk / (1024**3):.2f} GB")
        print(f"Disk Usage: {disk_usage:.2f}%")
    except (OSError, ValueError, ZeroDivisionError):
        print("Disk information unavailable")
```



```
    # Top processes by memory usage
    print("\nTOP 5 PROCESSES BY MEMORY USAGE")
    print("-" * 50)
    print(f"{'PID':<10}{'Memory (MB)':<15}{'Process'}")

    try:
        processes = []

        for pid in os.listdir("/proc"):
            if not pid.isdigit():
                continue

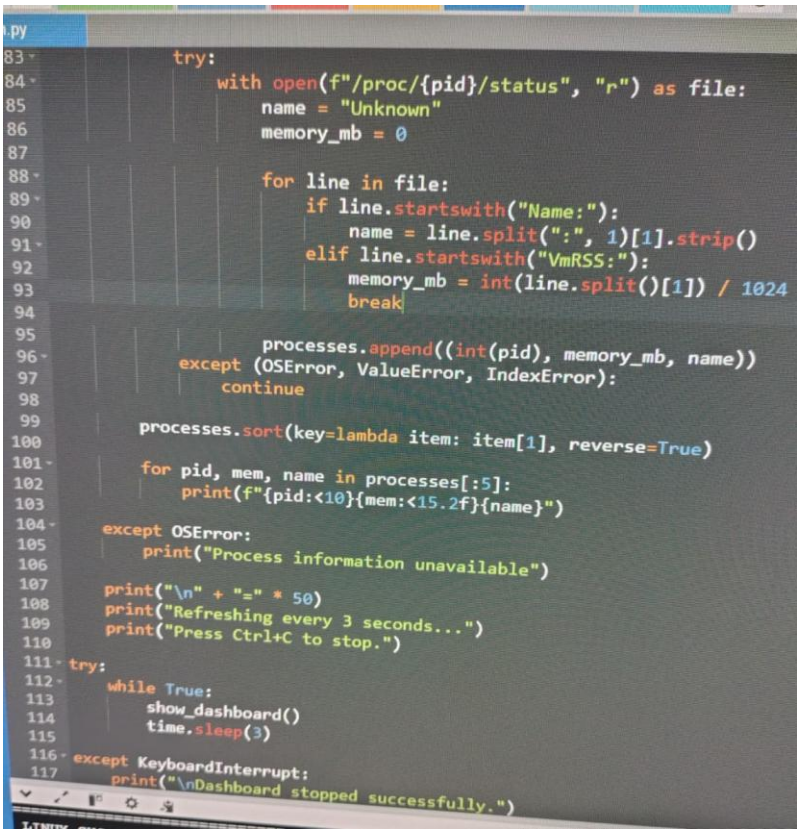
            try:
                with open(f"/proc/{pid}/status", "r") as file:
                    name = "Unknown"
                    memory_mb = 0

                    for line in file:
                        if line.startswith("Name:"):
                            name = line.split(":", 1)[1].strip()
                        elif line.startswith("VmRSS:"):
                            memory_mb = int(line.split()[1]) / 1024
                            break

                    processes.append((int(pid), memory_mb, name))
            except (OSError, ValueError, IndexError):
                continue

        processes.sort(key=lambda item: item[1], reverse=True)

        for pid, mem, name in processes[:5]:
            print(f"{pid:<10}{mem:<15.2f}{name}")
    except OSError:
```



```
.py
            try:
                with open(f"/proc/{pid}/status", "r") as file:
                    name = "Unknown"
                    memory_mb = 0

                    for line in file:
                        if line.startswith("Name:"):
                            name = line.split(":", 1)[1].strip()
                        elif line.startswith("VmRSS:"):
                            memory_mb = int(line.split()[1]) / 1024
                            break

                    processes.append((int(pid), memory_mb, name))
            except (OSError, ValueError, IndexError):
                continue

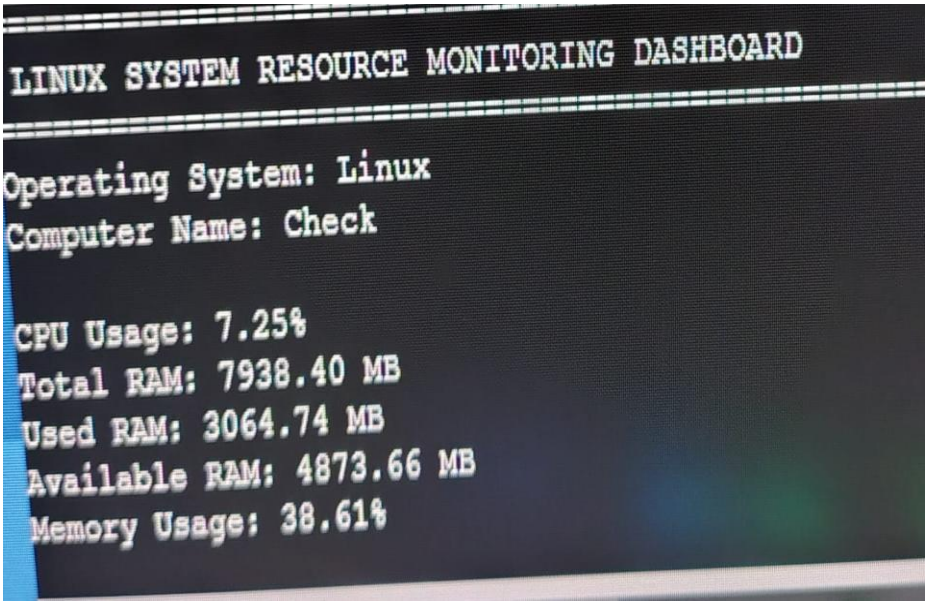
        processes.sort(key=lambda item: item[1], reverse=True)

        for pid, mem, name in processes[:5]:
            print(f"{pid:<10}{mem:<15.2f}{name}")
    except OSError:
        print("Process information unavailable")

    print("\n" + "=" * 50)
    print("Refreshing every 3 seconds...")
    print("Press Ctrl+C to stop.")

try:
    while True:
        show_dashboard()
        time.sleep(3)
except KeyboardInterrupt:
    print("\nDashboard stopped successfully.")
```

7. Output


LINUX SYSTEM RESOURCE MONITORING DASHBOARD
Operating System: Linux
Computer Name: Check
CPU Usage: 7.25%
Total RAM: 7938.40 MB
Used RAM: 3064.74 MB
Available RAM: 4873.66 MB
Memory Usage: 38.61%

8.Testing and result analysis

Test case	Expected result
Run the program	Dashboard displays system statistics
Open a CPU-intensive application	CPU usage may increase
Run several applications	Memory usage updates
Reach CPU usage of 80% or above	High CPU warning appears
Reach memory usage of 80% or above	High memory warning appears
Copy a large file	Disk I/O counters may increase
Press Ctrl + C	Monitoring terminates safely

9. Suggested 40–45 hour work plan

Activity	Hours
Study OS resource-monitoring concepts	6
Learn Linux commands and psutil	5
Design the dashboard and algorithm	4
Implement the monitoring modules	10
Testing and debugging	7
Analyze results and prepare screenshots	4
Prepare report and presentation	6
Total	42 hours

10.CO–PO–PSO Mapping

CO / PO	PO1	PO2	PO3	PO4	PO5
CO1	3	1			1
CO2	2	1	3		3
CO3	2	3	1	3	2
CO4	1	2	1	3	2
CO5	2	2	3	2	3

CO–PSO Mapping Table

CO / PSO	PSO1	PSO2
CO1	3	1
CO2	3	2
CO3	2	3
CO4	2	3
CO5	2	3

11. Conclusion

The Linux System Resource Monitoring Dashboard was successfully developed in Python. It reads CPU, memory, disk, and process information from the Linux /proc file system and displays it in a terminal dashboard that refreshes every 3 seconds. The output correctly showed the OS name, computer name, CPU usage, RAM statistics, and the top 5 memory-consuming processes. Through this project, we learned how the Linux kernel exposes system resource information and how process, CPU, memory, and file system concepts work in practice. In the future, the project can be extended with disk I/O monitoring, high usage warnings, graphical charts, and multi-computer monitoring.

12. References

1. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.
2. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2014.
3. M. Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010.
4. Linux manual page, “proc(5): process information pseudo-filesystem,” https://man7.org/linux/man-pages/man5/proc.5.html
5. Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/
6. psutil documentation, https://psutil.readthedocs.io